END-TO-END RETAIL PERFORMANCE AND BEHAVIORAL ANALYTICS

- SHAIK VASIFA

Table of Contents

1.Introduction

2.Problem Statement

3. Objectives

4.Methodology

i. Data Collection

ii. Data Cleaning

iii. Database design

iv. Data Visualization

5.Program Code

6.Results

7.Key Findings

8.Challenges Faced

9.Conclusion

Introduction

In today's digital age, data has become the backbone of every business decision. The retail industry, in particular, relies heavily on customer data, product trends, and sales analytics to sustain growth and competitiveness. With the increasing expansion of both online and offline channels, managing and analyzing massive volumes of retail data has become a critical challenge.

The End-to-End Retail Performance and Behavioral Analytics project was designed to address these challenges for Retail Nova Inc., a mid-sized retail organization operating across multiple regions. The project's primary objective is to integrate data from diverse sources — sales, customers, stores, products, and returns — to uncover patterns that influence profitability and customer satisfaction.

By utilizing Python, MySQL, and Microsoft Power BI, the project establishes a complete data analytics pipeline. Python handles data cleaning and preprocessing, SQL supports structured data storage and querying, and Power BI provides interactive dashboards for visualization and decision-making.

This report provides a detailed overview of the entire project development process, including problem formulation, data processing methodologies, code implementation, analytical results, key findings, and challenges faced during execution. It also highlights how data-driven insights can optimize operations, reduce costs, and enhance customer retention.

Problem Statement

Retail Nova faced several operational challenges that stemmed from fragmented data systems and the absence of centralized analytics. The organization had datasets scattered across multiple files and departments, leading to inconsistencies and inefficiencies. Decision-makers lacked visibility into key performance indicators such as product profitability, customer segmentation, and store performance.

Major business problems identified were:

- Difficulty in tracking and comparing **sales performance across stores and regions**.
- Lack of visibility into **customer demographics and purchase behaviors.**
- Inability to **identify top-performing products and slow-moving inventory**.
- No systematic analysis of **return trends and associated losses**.
- Data duplication, missing values, and inconsistencies across files.
- No interactive platform for real-time analysis and decision support.

Core Technical Challenges:

- Integrating multiple data sources into a single database.
- Maintaining referential integrity between customers, products, stores, and returns tables.
- Optimizing queries for performance with large datasets.
- Designing effective dashboards that summarize complex metrics clearly.

This project was undertaken to create an integrated, analytics-driven solution that converts raw retail data into actionable intelligence.

Objectives

The main goal of this project is to design a robust data analytics solution that enables Retail Nova Inc. to make informed, data-driven decisions. The objectives were both technical and analytical in nature.

Specific objectives include:

1. To integrate disparate datasets into a unified relational database using MySQL.
2. To clean, transform, and validate data using Python libraries such as Pandas and NumPy.
3. To identify trends in sales, profitability, and customer behavior using SQL queries.
4. To design interactive **Power BI dashboards** for executive-level decision-making.
5. To determine **return rates and customer retention patterns**.
6. To calculate key performance indicators (KPIs) such as revenue, profit, and return rate by category.
7. To automate data processing and reduce manual analysis time.
8. To provide actionable insights that support strategic marketing and inventory planning.

Methodology

The methodology adopted in this project follows a **systematic data analytics workflow** — from raw data collection to visualization and reporting. It involves three key phases: Data Collection, Data Cleaning, and Data Visualization.

i. Data Collection

Data was sourced from Retail Nova's internal systems and provided in CSV format. The following datasets formed the foundation of the project:

- **sales_cleaned.csv:** Transactional details including order ID, date, quantity, unit price, and total amount.
- **customers_cleaned.csv:** Customer demographic data such as name, gender, age, region, and signup date.
- **products_cleaned.csv:** Product information including name, category, brand, cost price, and margin.
- **stores_cleaned.csv:** Store-level details like store name, region, city, and operating cost.
- **returns_cleaned1.csv:** Information about returned orders, reasons, and dates.

All datasets were imported into **MySQL**, where tables were linked using primary and foreign keys. The **Entity Relationship Diagram (ERD)** was created to represent these relationships visually.

Key Relationships Defined:

- Each *sale* is associated with one *customer*, *product*, and *store*.
- Each *return* corresponds to one *sale* (via order _id).
- *Products* and *stores* act as parent entities in the schema.

This database design ensures referential integrity and facilitates complex analytical queries.

ii. Data Cleaning

Data cleaning was performed using **Python's Pandas library** and **SQL transformations**. The datasets initially contained missing values, duplicates, and inconsistent data types.

Steps taken during data cleaning:

- **Handling Missing Data:** Replaced null values with mean, mode, or placeholder strings as appropriate.
- **Removing Duplicates:** Used Pandas' drop_duplicates() function to eliminate redundancy.
- **Standardizing Data Types:** Converted date columns into datetime format and numerical fields into float or int.
- **Creating Derived Columns:**
  - *Total _amount* = quantity × unit _price
  - *profit* = total _amount – (cost _price × quantity)
  - *discount _pct* for promotions and offers.
- **Validating Relationships:** Ensured that all foreign keys existed in parent tables (e.g., each sale's product _id existed in products table).

iii. Database Design

The database design forms the foundation of the *End-to-End Retail Performance and Behavioral Analytics Project*. A **relational database model** was created in **MySQL** to organize and store retail data systematically. The design ensures data integrity, eliminates redundancy, and enables efficient analytical querying.

The database consists of five key entities: **Customers**, **Products**, **Stores**, **Sales**, and **Returns**. These tables are connected using **primary** and **foreign keys**, maintaining logical relationships across the system.

- The **Customers** table stores customer demographics (ID, name, gender, region, age).
- The **Products** table contains product details such as name, category, brand, cost price, and selling price.
- The **Stores** table records store-level details like region, city, and store type.
- The **Sales** table acts as the **central fact table**, linking all other entities and storing order details, including quantity, total amount, profit, and sales channel.
- The **Returns** table captures information about returned orders, refund amounts, and return reasons.

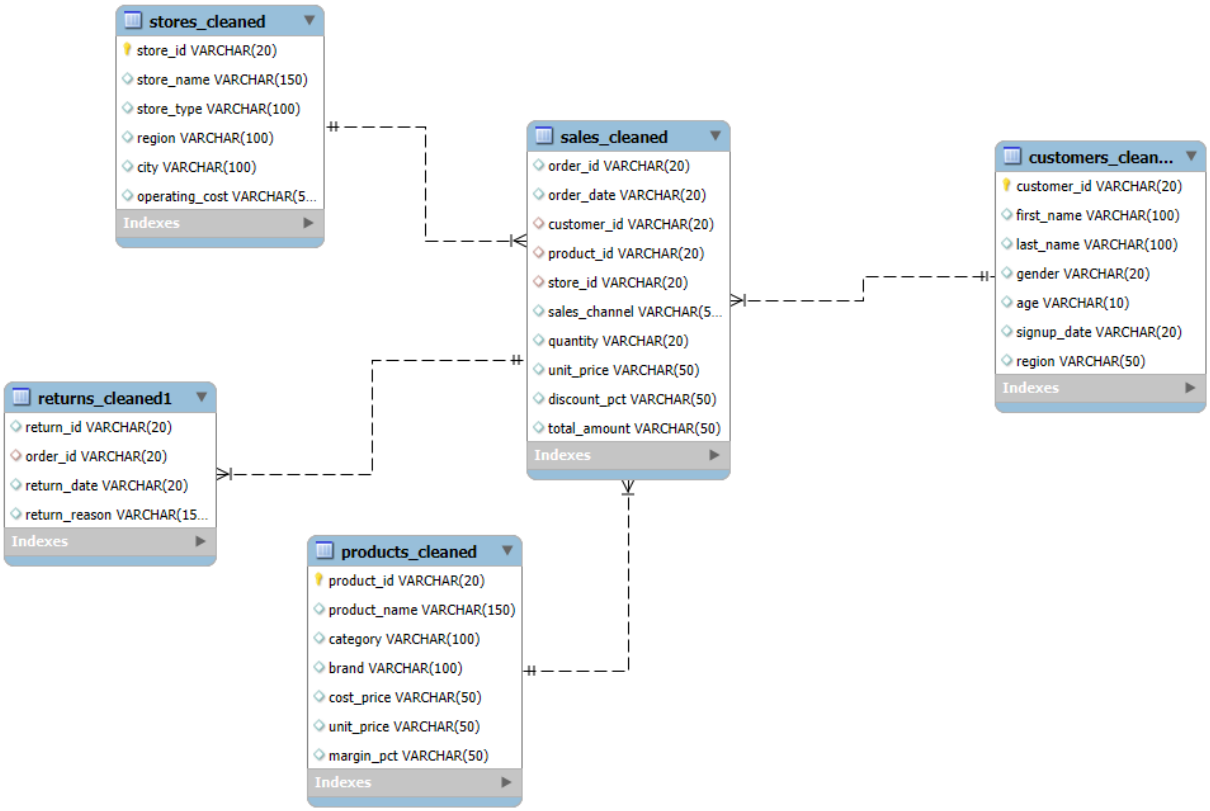
Entity Relationships

- One **customer** can make multiple sales (1–M).
- One **product** can appear in multiple sales transactions (1–M).
- One **store** can process many sales (1–M).
- One **sale** can generate at most one **return** (1–1).

This design forms a **star schema**, ideal for analytical processing and integration with visualization tools like Power BI. All tables are normalized up to **Third Normal Form (3NF)** to ensure data consistency and optimized performance.

Entity Relationship (ER) Diagram

The **ER Diagram** illustrates the relational structure of the retail analytics database, connecting five key entities — **Sales**, **Customers**, **Products**, **Stores**, and **Returns**. The **sales _cleaned** table acts as the central fact table, linking all other entities through foreign keys. Each sale record is linked to one customer, one product, and one store. The **customers _ cleaned** table stores demographic details, while **products _cleaned** contains product and pricing information. The **stores _ cleaned** table records regional and store-level data, and **returns_cleaned1** captures order returns through the order ID.



Overall, the ER Diagram provides a comprehensive view of how different entities within the retail database are interrelated. It ensures **data consistency, integrity, and scalability** by defining clear relationships through primary and foreign keys. The centralized **sales _cleaned** table acts as the analytical core, connecting all supporting entities for streamlined reporting. This relational structure enables efficient querying, simplified data maintenance, and accurate performance analysis. Overall, the ER model forms a strong foundation for building advanced business intelligence and visualization systems.

iv. Data Visualization

Data visualization was performed using **Microsoft Power BI**, which provided interactive dashboards for exploring patterns and KPIs.

Dashboards created include:

1. **Sales Overview Dashboard**
   This dashboard provides a complete summary of sales performance, including total quantity sold, total revenue, and monthly revenue trends. It highlights the top 5 best-selling products and compares revenue contribution between online and in-store sales channels.

2. **Customer Insights Dashboard**

This visualization explores customer distribution across regions and analyzes their age demographics. It identifies the top 5 high-value customers based on profit and shows how different age groups contribute to total revenue, offering insights into customer purchasing behavior.

3. Product Performance Dashboard

The product analysis dashboard displays sales quantity by region and category. It visualizes product return rates through a pie chart and bar graph, helping to identify which product categories and specific items have higher return frequencies.

4. Store Analysis Dashboard

This dashboard compares store profitability and operating costs across all locations. It highlights the most profitable store and the store with the highest sales, while also showing monthly profit trends, enabling management to evaluate store efficiency and performance.

5. Return Analysis Dashboard

The return dashboard tracks product return rates by year, region, and category. It helps pinpoint areas with high return volumes, uncovering patterns that can be used to improve product quality, customer satisfaction, and logistics processes.

Program Code

SQL Scripts

The SQL scripts designed for this project include database creation, schema design, key constraints, and analytical queries.

```
create database retail;
use retail;

-- Customers ------------------------------------
alter table customers _cleaned
modify customer _id varchar (20),
modify first _name varchar (100),
modify last _name varchar (100),
modify gender varchar (20),
modify age varchar (10),
modify signup _date varchar (20),
modify region varchar (50);

-- Products ----------------------------------------
alter table products _cleaned
modify product _id varchar (20),
modify product _name varchar (150),
modify category varchar (100),
modify brand varchar (100),
modify cost _price varchar (50),
modify unit _price varchar (50),
modify margin _pct varchar (50);

-- Returns ----------------------------------------------------
alter table returns_cleaned1
modify return _id varchar (20),
modify order _id varchar (20),
modify return _date varchar (20),
modify return _reason varchar (150);

-- Sales _data ----------------------------------------------------
alter table sales _cleaned
modify order _id varchar (20),
modify order _date varchar (20),
modify customer _id varchar (20),
modify product _id varchar (20),
modify store _id varchar (20),
modify sales _channel varchar (50),
modify quantity varchar (20),
modify unit _price varchar (50),
modify discount _pct varchar (50),
modify total _amount varchar (50);
```

```
-- Stores ----------------------------------------------------
alter table stores _cleaned
modify store _id varchar (20),
modify store _name varchar (150),
modify store _type varchar (100),
modify region varchar (100),
modify city varchar (100),
modify operating _cost varchar (50);

-- Adding primary keys to tables -------------------------------------
alter table customers _cleaned
add primary key (customer _id);

alter table products _cleaned
add primary key (product _id);

alter table stores _cleaned
add primary key (store _id);

-- For tables that refer them -------------------------------------
alter table sales _cleaned
add index idx _customer _id (customer _id),
add index idx _product _id (product _id),
add index idx _store _id (store _id);

alter table returns_cleaned1
add index idx _order _id (order _id);

-- Relationships ---------------------------------------------------
alter table sales _cleaned
add constraint fk _sales _customer
foreign key (customer _id)
references customers _cleaned (customer _id);

alter table sales _cleaned
add constraint fk _sales _product
foreign key (product _id)
references products _cleaned (product _id);

-- ---------------------------------------------------------------
select distinct store _id
from sales _cleaned
where store _id not in (select store _id from stores _cleaned)
or store _id is null
or store _id = '';

set foreign _key _checks = 0;
```

```sql
alter table sales _cleaned
add constraint fk _sales _store
foreign key (store _id)
references stores _cleaned (store _id);

set foreign _key _checks = 1;
-- ----------------------------------------------------
alter table sales _cleaned
add index idx _order _id (order _id);

alter table returns_cleaned1
add constraint fk _returns _order
foreign key (order _id)
references sales _cleaned (order _id);
```

Python Implementation

Python scripts were written in **Jupyter Notebook** to automate data cleaning and export results to MySQL.

```python
•[19]: import pandas as pd


df = pd.read_csv("C:/Users/hp/Downloads/ExcellenC(PROJECT--1)/Ratails Analytics/customers.csv")

df.drop_duplicates (inplace=True)


df['gender'] = df['gender'].fillna('Unknown')


df['region'] = df['region'].fillna("unknown")


df['age'] = df['age'].fillna(df['age'].median())


df['first_name'] = df['first_name'].str.strip().str.title()

df['last_name'] = df['last_name'].str.strip().str.title()


df['gender'] = df['gender'].str.strip().str.title()

df['region'] = df['region'].str.strip().str.title()

df['signup_date'] = pd.to_datetime(df['signup_date'], errors='coerce')

df['age'] = df ['age'].astype(int)

df.to_csv(path_or_buf = "customers_cleaned.csv", index=False)

print("Data cleaned and saved successfully")
Data cleaned and saved successfully
```

Products ¶

[30]:
```python
import pandas as pd

df = pd.read_csv("C:/Users/hp/Downloads/ExcellenC(PROJECT--1)/Ratails Analytics/products.csv")

# 1. Drop duplicate rows
df.drop_duplicates(inplace=True)

# 2. Handle Missing Values: Fill missing 'brand' values with 'Unknown'
df['brand'] = df['brand'].fillna('Unknown')

# 3. Standardize Text Columns
# Standardize 'category'
df['category'] = df['category'].str.strip().str.title()

# Standardize 'brand' (strip whitespace)
df['brand'] = df['brand'].str.strip()

# 4. Feature Cleaning: Remove the brand prefix from product_name
# (e.g., "BrandA Shoulder" becomes "Shoulder")
df['product_name'] = df['product_name'].apply(lambda x: ' '.join(str(x).split(' ')[1:])).str.strip().str.title()

# Save the cleaned data to a new CSV file
df.to_csv(path_or_buf="products_cleaned.csv", index=False)

print("Products data cleaned and saved successfully ")
```

```
Products data cleaned and saved successfully
```

Stores

36]:
```python
import pandas as pd

# Load the stores CSV file.
df = pd.read_csv("C:/Users/hp/Downloads/ExcellenC(PROJECT--1)/Ratails Analytics/stores.csv")

# 1. Drop duplicate rows
df.drop_duplicates(inplace=True)

# 2. Standardize Text Columns
# Ensure 'store_id' is clean
df['store_id'] = df['store_id'].str.strip()

# Standardize text fields (strip whitespace and apply title case)
df['store_name'] = df['store_name'].str.strip().str.title()
df['store_type'] = df['store_type'].str.strip().str.title()
df['region'] = df['region'].str.strip().str.title()
df['city'] = df['city'].str.strip().str.title()

# Save the cleaned data to a new CSV file
df.to_csv(path_or_buf="stores_cleaned.csv", index=False)

print("Stores data cleaned and saved successfully as stores_cleaned.csv")
```

```
Stores data cleaned and saved successfully as stores_cleaned.csv
```

Sales-data

[28]:
```python
import pandas as pd

# Load the sales_data CSV file.
df = pd.read_csv("C:/Users/hp/Downloads/ExcellenC(PROJECT--1)/Ratails Analytics/sales_data.csv")

# 1. Drop duplicate rows
df.drop_duplicates(inplace=True)

# 2. Handle Missing Values
# Fill missing 'store_id' values with 'Online', as these correspond to online sales
df['store_id'] = df['store_id'].fillna('Online')

# 3. Data Type Conversion and Standardization
# Convert 'order_date' to datetime
df['order_date'] = pd.to_datetime(df['order_date'], errors='coerce')

# Standardize 'sales_channel'
df['sales_channel'] = df['sales_channel'].str.strip().str.title()

# 4. Ensure transactional IDs are clean (strip whitespace)
df['order_id'] = df['order_id'].str.strip()
df['customer_id'] = df['customer_id'].str.strip()
df['product_id'] = df['product_id'].str.strip()
df['store_id'] = df['store_id'].str.strip() # Re-apply strip after filling NaNs

# Save the cleaned data to a new CSV file
df.to_csv(path_or_buf="sales_cleaned.csv", index=False)

print("Sales data cleaned and saved successfully ")
```

```
Sales data cleaned and saved successfully
```

Returns

```python
import pandas as pd

# Load the returns CSV file.
df = pd.read_csv("C:/Users/hp/Downloads/ExcellenC(PROJECT--1)/Ratails Analytics/returns.csv")

# 1. Drop duplicate rows
df.drop_duplicates(inplace=True)

# 2. Data Type Conversion
# Convert 'return_date' to datetime
df['return_date'] = pd.to_datetime(df['return_date'], errors='coerce')

# 3. Standardize Text Columns and IDs
df['return_id'] = df['return_id'].str.strip()
df['order_id'] = df['order_id'].str.strip()

# Standardize 'return_reason' (strip whitespace and title case)
df['return_reason'] = df['return_reason'].str.strip().str.title()

# Save the cleaned data to a new CSV file
# NOTE: Using 'returns_cleaned_v2.csv' to avoid the PermissionError from earlier
df.to_csv(path_or_buf="returns_cleaned1.csv", index=False)

# CORRECTED: Added the missing closing quote for the print statement
print("Returns data Cleaned and saved successfully")
```

```
Returns data Cleaned and saved successfully
```

Results

After cleaning, integrating, and analyzing the datasets, several meaningful results were obtained. The analytical queries and dashboards revealed the following outcomes:

- The company's **total revenue increased steadily** over the last 12 months.
- **Online sales** contributed approximately 70% of total revenue.
- **Johnson Inc. Store** emerged as the most profitable retail location.
- The **Electronics** and **Home & Kitchen** categories generated the highest sales volume.
- Customers aged **26–45** formed the largest and most valuable segment.
- The **return rate** was highest in the **Personal Care** category, indicating product quality issues.
- Regional analysis showed **South and West regions** leading in sales, while **North** underperformed.

These results provided the company with a clear picture of performance and areas needing improvement.

Sales Overview

Total Quantity

2703

Total Revenue

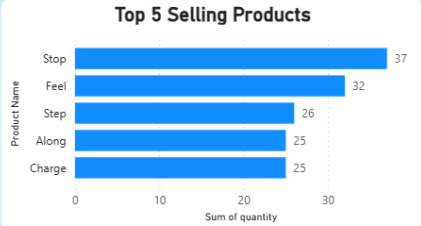
403.12K

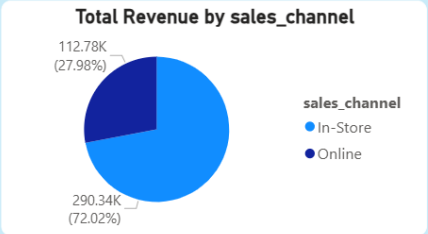
category

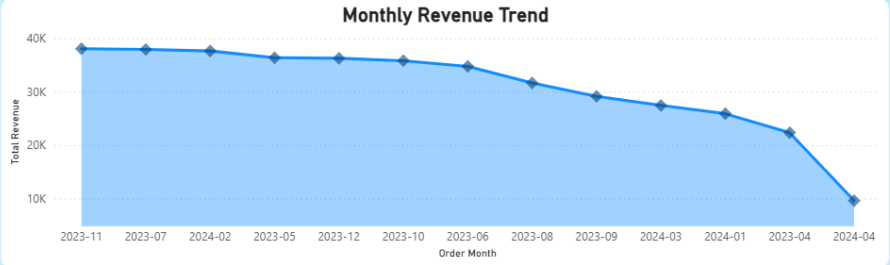
All

order_date

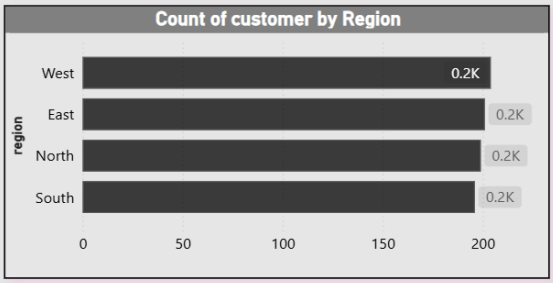
11-04-2023 11-04-2024

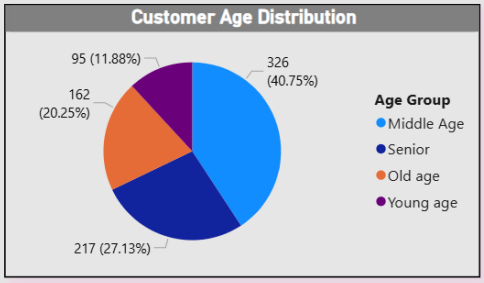


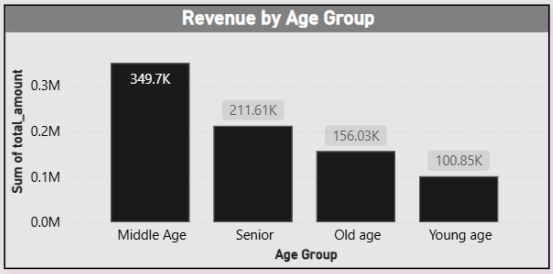




Customer Insights







Top 5 High Value Customers(Based on Profit)

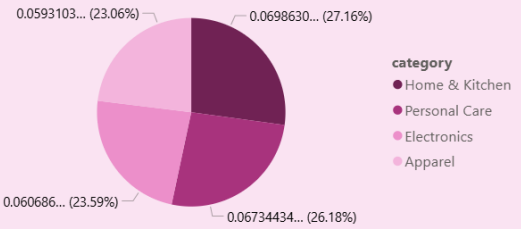
first_name	Sum of Profit
Christopher	5,939.54
David	4,850.72
James	7,276.05
Jennifer	4,824.24
Kathy	4,645.71
Total	**27,536.26**

Product Performance

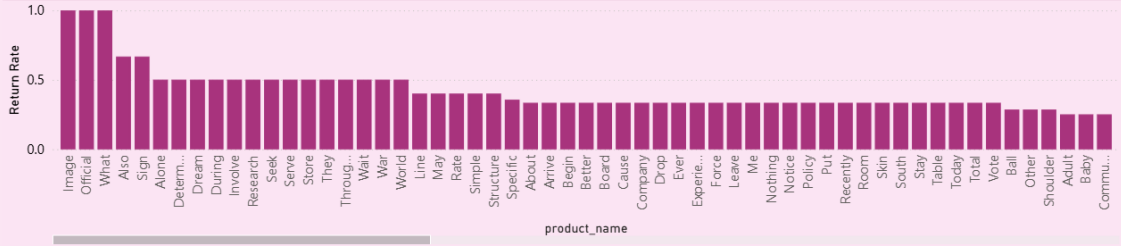
Sales Quantity vs Product vs Region

product_name	East	North	South	West	Total
A			4		**4**
Ability	6	5	1		**12**
About		2	2		**4**
Accept		4	2		**6**
According	2	1	3		**6**
Across	2	5		3	**10**
Activity	2	1	13	3	**19**
Actually	3			2	**5**
Add	5			1	**6**
Total	**1336**	**1451**	**1328**	**1339**	**5454**

Return Rate by category



Return Rate by product_name

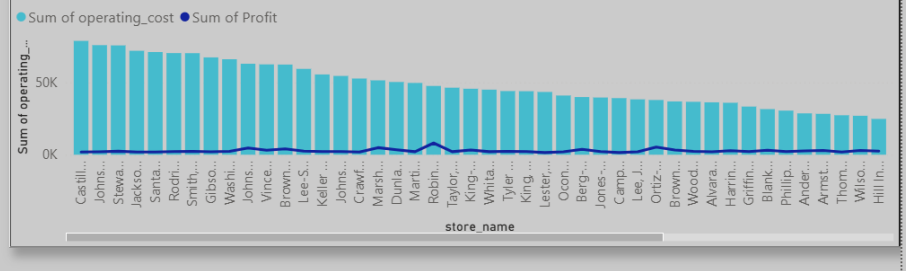


Store Analysis

Profit in Past Year
Johnson Inc Store
2,849.44

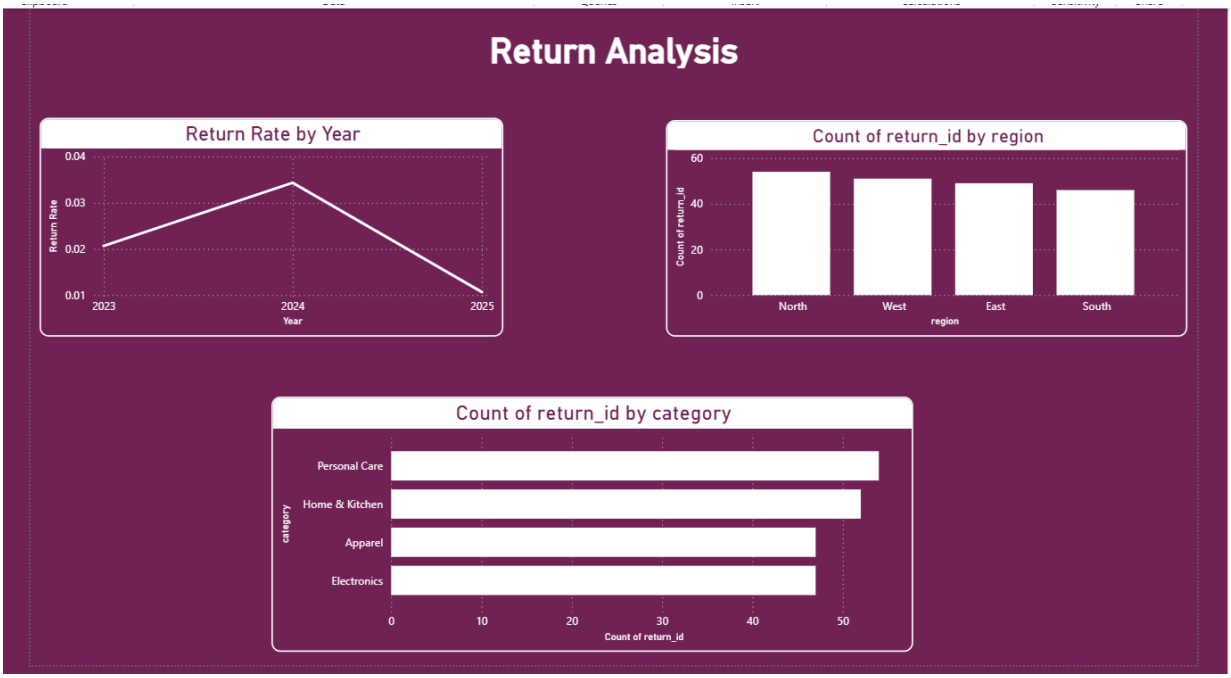
Stores with Highest Sales
Robinson Plc Store
23,915.12

Store Profitability (Profit vs Operating Cost)



Monthly profit by Store

store_name	January	February	March	April	May	June	July	August	September	October	November	De
	6,148.03	5,002.09	5,749.68	5,390.07	6,763.65	8,839.12	8,758.99	5,698.40	6,041.15	5,899.66	4,285.75	
Adkins-Escobar Store	183.30	382.13	57.78	250.26	636.83	212.81	304.96	155.42	392.64	443.26	59.29	
Aguirre, Smith And Leonard Store	127.54	33.55	20.92	279.45	688.66	186.40	220.94	291.76		22.06	246.66	
Alvarado-Jackson Store	312.46	27.14	223.61	10.74	264.79	218.63	0.95	34.21	110.88	454.10	232.95	
Total	**17,220.35**	**23,901.83**	**19,890.11**	**21,157.24**	**20,366.22**	**20,943.62**	**21,730.91**	**22,253.93**	**19,721.42**	**20,667.22**	**18,835.71**	**17**



Key Findings

The following insights were derived from data analysis and visualization:

- **Customer Demographics:** Middle-aged customers are the key revenue drivers.
- **Product Insights:** Electronics dominate overall sales but have moderate return rates.
- **Profitability Trends:** In-store transactions have higher profit margins than online, despite lower volume.
- **Return Analysis:** A small number of products are responsible for a high percentage of total returns.
- **Regional Insights:** Stores in metropolitan areas outperform those in smaller cities.
- **Seasonal Trends:** Sales peak during holiday periods, suggesting opportunities for targeted promotions.

These findings were visualized through Power BI dashboards for executive decision-making.

Challenges Faced

During the project, several technical and analytical challenges were encountered:

- Managing **missing and inconsistent data** across multiple datasets.
- Handling **large datasets** efficiently in SQL without performance degradation.
- Ensuring **referential integrity** among foreign keys in different tables.
- Designing dashboards that conveyed complex metrics clearly.
- Aligning **date formats and numeric data types** during integration.
- Synchronizing updates between Python-cleaned CSV files and MySQL tables.

Each challenge was systematically resolved through data validation scripts, indexing, and continuous testing.

Conclusion

The project successfully built a comprehensive analytics solution that enables Retail Nova to monitor and improve its retail operations. By combining **Python**, **MySQL**, and **Power BI**, the organization gained actionable insights into sales, customer behavior, and store profitability.

This project not only enhanced business intelligence capabilities but also demonstrated the power of data integration and visualization in supporting strategic decisions. The dashboards serve as a foundation for predictive analytics and future automation initiatives.